

Carga Electrónica Programable

Gomez Pablo Sebastián

Técnicas Digitales III, UTN FRA

Avellaneda, Argentina

pablogomez50@gmail.com

Abstract— El proyecto desarrollado consiste en una carga electrónica programable con Raspberry Pi Pico 2 (RP2350) ejecutando el SO FreeRTOS (v202210.01 LTS). Utilizando el sensor INA219 para mediciones de tensión y corriente, aplicando un control PID para la polarización de un MOSFET IRF540 con la técnica de modulación de ancho de pulso (PWM) filtrado. Implementación de datalogger con reloj en tiempo real (RTC) con almacenamiento en memoria SD. Se programó en lenguaje C con el entorno de desarrollo IDE Visual Studio Code v1.103.2 y Pico SDK v2.1.1. Integración por medio de driver UART con una aplicación de usuario desarrollada en un sistema basado en Linux, kernel v6.12.47.

Palabras clave: Carga electrónica - FreeRTOS - Resistencia Constante - INA219 - IRF540N - Memoria SD - RP2350 - RPI 5 - Linux - GPOS - Char device

ÍNDICE

I. FUNDAMENTACIÓN.....	1
A. Problemática.....	1
B. Solución.....	1
C. Fundamentación Software.....	1
D. Fundamentación Hardware.....	1
II. OBJETIVOS.....	2
III. ALCANCE.....	2
IV. DIAGRAMA DE BLOQUES.....	2
V. DIAGRAMA DE SOFTWARE.....	2
VI. DIAGRAMA DE TAREAS DE EGA.....	2
VII. COMANDOS DE USUARIO GPOS.....	3
VIII. CONCLUSIONES.....	3
IX. REFERENCIAS.....	4
X. ANEXOS.....	5

I. FUNDAMENTACIÓN

A. Problemática

En el diseño, fabricación y/o prueba de funcionamiento de fuentes de alimentación y baterías se debe verificar su estabilidad, regulación y protección bajo una variedad de cargas.

Contar con una variedad de resistencias fijas es impráctico, debido a los distintos valores y potencias necesarias para el ensayo.

B. Solución

Frente a esta problemática, se utilizan cargas electrónicas, dispositivos que proveen una resistencia constante programable, independientemente de la tensión a la que esté sometido, informando los valores de tensión, corriente y resistencia por medio de un display LCD, o luego del ensayo, observar los datos

almacenados en formato csv. Además, cuenta también con protecciones contra sobretensión, sobrecorriente y sobrecalentamiento, aumentando así su confiabilidad.

El mismo puede ser programado para que presente valores de resistencia constantes, en forma de escalón, o una variación en forma de rampa.

Este dispositivo, al conectarse con una Raspberry Pi habilita el control remoto del dispositivo por medio de una aplicación de consola disponible a través de ssh, donde se puede controlar variables del dispositivo, o por medio de una aplicación web, donde se puede controlar las variables y adquisición de datos de funcionamiento.

C. Fundamentación Software

Para la realización de este proyecto se emplea el uso de un sistema operativo en tiempo real, específicamente, FreeRTOS[1], organizando el dispositivo en rutinas/tareas, asignando niveles de prioridad y flujo de datos por medio de colas.

La configuración y el control del dispositivo se puede realizar por medio de dos botones y un encoder mecánico o, una página web corriendo sobre una Raspberry Pi 5, configurando de forma remota el dispositivo, por medio del puerto UART0 del RP2350, gestionado por interrupciones de lectura que escriben a una queue `queue_uart_rx`, procesada por una tarea `task_uart` y la escritura gestionando por un mutex `uart_tx_mutex`.

La gestión del bus de I2C1 se vuelve relevante al tener tareas y dispositivos varios que consumen el mismo recurso, tales como el sensor INA219 para mediciones de corriente y tensión, el módulo RTC para el tiempo, y para la interfaz gráfica con la pantalla LCD 16x2. Se implementó una tarea guardiana en la que se distribuye y administra el consumo del bus de comunicación.

Para la comunicación con la memoria SD, se utilizó la librería FatFS[2], que provee un sistema de archivos montado sobre la interfaz SPI.

La medición de temperatura se realiza por medio del canal ADC0 de forma analógica, con un diodo 1N4148 linealizado por una fuente de corriente de 0.1mA, obteniendo una tensión proporcional a la temperatura que es amplificada al rango de trabajo.

D. Fundamentación Hardware

Este proyecto se basará en el microcontrolador RP2350, doble núcleo Cortex-M33, en la placa de desarrollo Raspberry Pi Pico 2[3]. Debido a su alta frecuencia de trabajo de 150 MHz, alto poder de procesamiento y control sobre los periféricos tales como PWM, I2C, SPI, GPIO y ADC.

Para poder desarrollar la carga electrónica programable es necesario tener un componente que presente una resistencia que se mantenga fija, teniendo una respuesta directa a una señal de salida de nuestro sistema digital, con una sensibilidad baja a perturbaciones externas tales como variaciones de tensión o corriente del sistema a ensayar. Los componentes que cumplen con

estos requisitos son los transistores de tipo MOSFET, que presentan una resistencia de salida dependiendo de la tensión aplicada en el gate del mismo.

Para la interfaz de usuario se decidió utilizar un encoder mecánico con botón incluido, dos botones y display LCD 16x2.

Frente a la ausencia de un DAC en el microcontrolador RP2350, se optó por utilizar una salida PWM con un filtro activo pasabajos de 2do orden Sallen-Key con una frecuencia de corte de 507 Hz. A partir de la señal de salida se amplifica y se suma un offset con el objetivo de aumentar la resolución y reducir el efecto de la tensión umbral de compuerta del MOSFET.

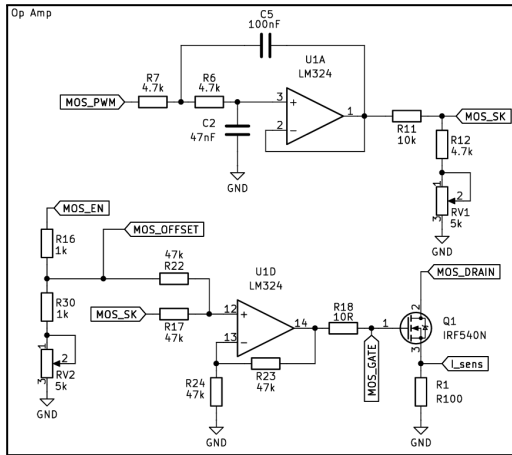


Figura 1. Etapa de driver MOSFET.

En el caso de dispositivo utilizando el sistema operativo de propósito general (GPOS), se necesita la posibilidad de tener pines expuestos para realizar la comunicación a través de UART, por lo que se utilizó una Raspberry Pi 5 de 8GB.

II. OBJETIVOS

- **Simular una carga** en un circuito eléctrico, absorbiendo energía de una fuente de alimentación o dispositivo bajo prueba de forma controlada y ajustable.
- **Medir y mostrar la resistencia** de la carga electrónica.
- **Mostrar y modificar el valor de resistencia** que presenta la carga electrónica, manteniendo constante dicho valor.
- **Presentar dos modos de funcionamiento**, uno por escalón y otro por rampa, los cuales son configurables para el usuario.
- **Proteger el MOSFET** de valores de sobretensión, sobrecorriente y sobret temperatura.
- **Obtener los valores significativos** del ensayo de forma sencilla, para su futuro análisis.

III. ALCANCE

Se pretende lograr una carga electrónica con los siguientes parámetros:

- **Tensiones de entrada (Vin):** 1.2-12 volts.
- **Corrientes de entrada (Iin):** 1-250 mA
- **Resistencia efectiva:** 20-2000 Ω , error menor al 5%.

IV. DIAGRAMA DE BLOQUES

El diagrama de la figura 2 muestra las interconexiones entre los módulos y la Raspberry Pi Pico 2.

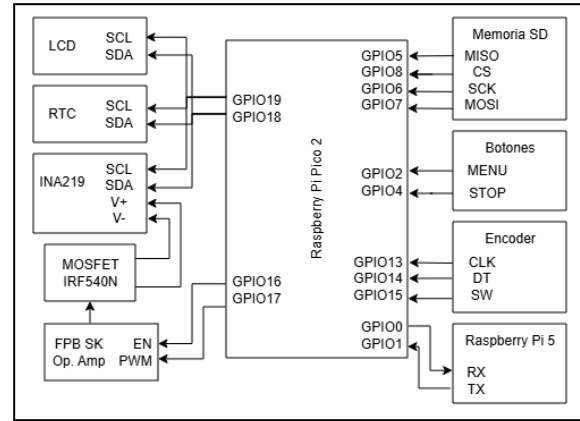


Fig. 2 Diagrama de bloques de la carga electrónica programable.

V. DIAGRAMA DE SOFTWARE

El sistema está compuesto por un módulo de kernel en el cual se inicializa el driver de UART y 2 char devices, egb_commands y egb_data, además de gestionar los strings de texto en dos listas, msg_list_cmd y msg_list_data por medio de un identificador al inicio del mensaje. Se utilizaron listas de linux, utilizando como queues para evitar una pérdida de datos y una estandarización con los char devices separando el stream de datos de los comandos.

Se crearon dos aplicaciones en user space, en primera medida una app de cli para poder obtener y setear las variables, iniciar y frenar el control PID. Como segunda app se creó un backend en FastAPI para poder utilizar una app web construida en Next.js, donde poder realizar las mismas acciones que la app de cli, además de un websocket para un stream de datos del dispositivo, con un hilo consumiendo el char dev egb_data.

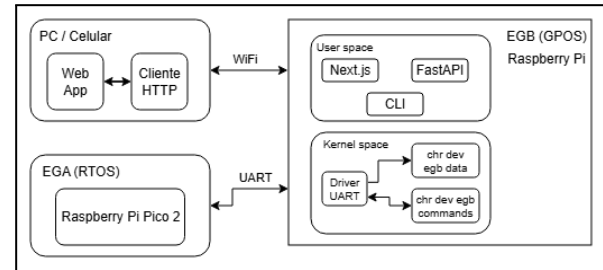


Fig. 3 Diagrama de software de la egb.

VI. DIAGRAMA DE TAREAS DE EGA

En la Fig. 4 se muestra el diagrama de tareas implementado en el microcontrolador RP2350, se utiliza FreeRTOS.

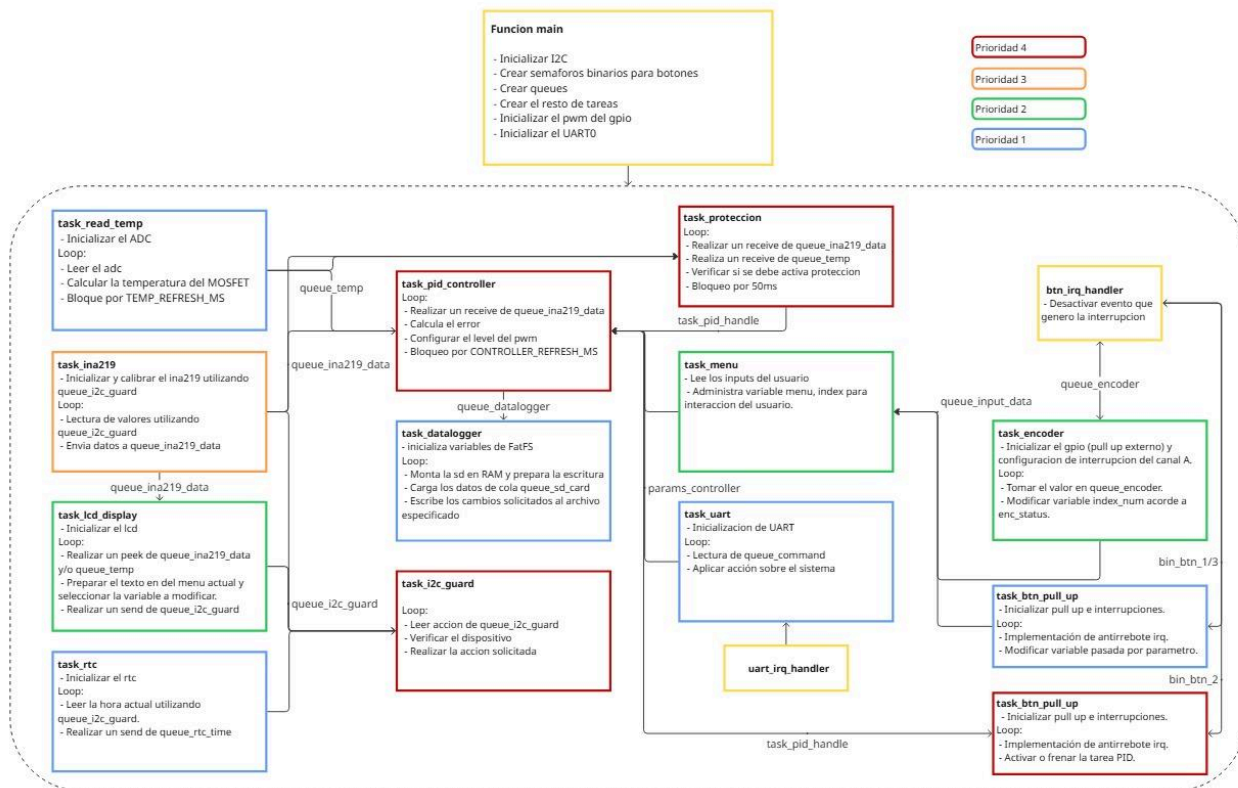


Figura 4. Diagrama de tareas FreeRTOS

VII. COMANDOS DE USUARIO GPOS

Para interactuar con el sistema corriendo FreeRTOS, y únicamente obtener información, se utilizan los siguientes comandos:

- \$get protection: indica si hay una protección que se activó.
- \$get status: mostrar si el sistema de control se encuentra en ejecución o no.
- \$get sd_card: indica el estado de conexión de la memoria SD.
- \$get voltage: tensión de entrada.
- \$get current: corriente de entrada.
- \$get temp: temperatura.

Para las variables de escritura y lectura se utilizan los siguientes formatos:

\$get <var name> / \$set <var name> <value>

- kp: constante PID proporcional.
- ki: constante PID integrativa.
- kd: constante PID derivativa.
- r_target: resistencia objetivo.
- t_target: tiempo hasta que se llega al objetivo, siendo 0 un estímulo escalón y un número mayor a 0 una entrada rampa.
- int_lim: máximo valor del término integrativo.
- der_lim: máximo valor del término derivativo.
- max_temp: temperatura a la cual se activa la protección.
- max_current: corriente a la cual se activa la protección.
- max_voltage: tensión a la cual se activa la protección.

La tarea `task_datalogger` se encarga de realizar la conexión con

la memoria sd y montar el disco para poder leer y escribir. Una vez montado intenta leer un archivo de configuración para tomar los valores de seteo previos. Luego, se bloquea a la espera de la cola `queue_sd_card`, una vez recibida genera el archivo objetivo completando la información en instrucción de `append` si es `log` de información y de sobrescritura si es de configuración.

VIII. CONCLUSIONES

Frente a los objetivos especificados en la sección de objetivos y alcances, se logró una correcta operación del transistor MOSFET como dispositivo de carga electrónica, dentro de la región de trabajo seleccionada.

Las protecciones especificadas se logran detectar y proteger, además de la posibilidad de regular estos límites dependiendo el objetivo de la medición a realizar.

Con respecto a la resistencia efectiva obtenida por el dispositivo, se pudo cumplir con los valores seteados por el rango de valores esperado.

Se contó con una mejora en el rango de trabajo al añadir el offset y el filtro de segundo orden para la gestión de la tensión V_{gs} controlada por el PWM.

IX. REFERENCIAS

- [1] FreeRTOS: <https://www.freertos.org/>
 [2] FatFS: <https://elm-chan.org/fsw/ff/>
 [3] Raspberry Pi Pico 2:
<https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf>

X. ANEXOS

Anexo 1 - App consola

Se adjuntan captura de los menús de la app CLI:

```
-----{ Aplicación UART EGB }-----
Ingrese accion a realizar:
-----
1> Enviar configuración
2> Obtener información
3> Iniciar PID
4> Frenar PID
5> Leer uart
6> Salir
-----
Opción [1-6]:
```

Figura 5. Menú principal

Anexo 2 - Página Web

Se adjuntan capturas de la UI de la página web:

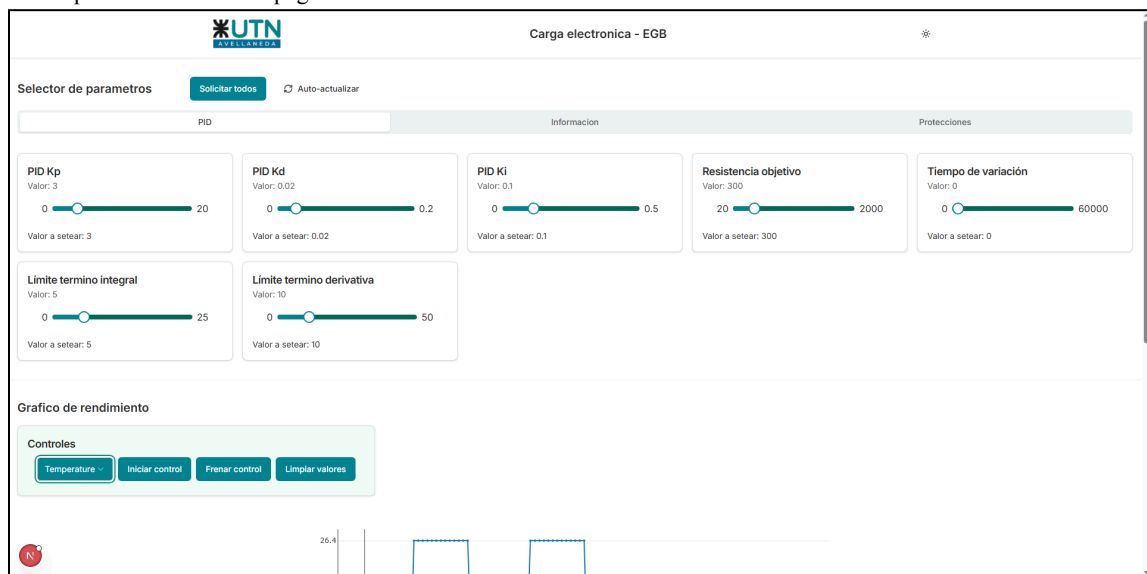


Figura 6. Selector de parámetros página web

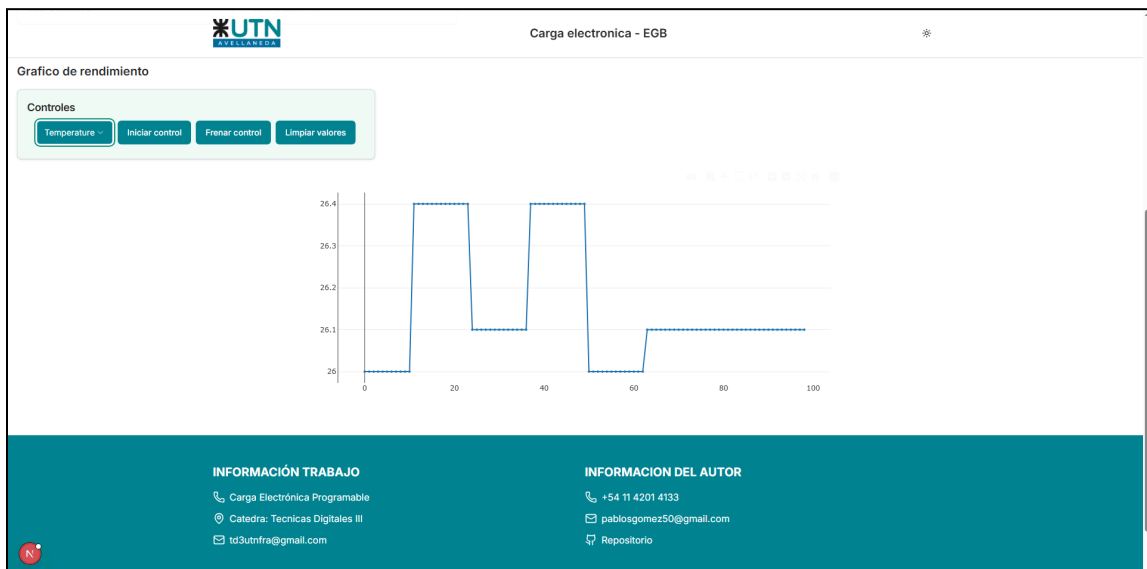


Figura 7. Gráfico de datos en tiempo real